

Informe Técnico

Autores: Alejandro Rojas Serna – Ángel Torres Panesso

Curso: Análisis y Desarrollo de Algoritmos ADA

Resumen

Este proyecto implementa una red neuronal **MLP** con una capa oculta desde cero (únicamente con **Numpy**), entrenada mediante retro propagación y **SGD**. Se aplica la clasificación de imágenes (**MNIST** y **Digits**) y, opcionalmente, a tablas simples (iris y Wine) para demostrar versatilidad. El trabajo se organiza en cuatro fases: RA1 (Diseño y complejidad del **MLP**), RA2 (Algoritmos de selección y ordenamiento y método maestro), RA3 (Estructuras de datos avanzadas) y RA4 (baseline K-NN y verificación de gradientes). Se presentan derivaciones asintóticas, validaciones y comparativas de el rendimiento. Se alcanza la precisión del 100% en entrenamiento para **MNIST** y **Digits** con 5000 Épocas, superando el 85% exigido.

Notas: **Baseline** es la red neuronal entrenando sin **hard minig** es decir usa todas las muestras sin filtrar las de mayor perdido.

K-NN (k-vecinos más cercanos) es un clasificador simple que no entrena pesos, simplemente guarda todos los datos de entrenamiento. Cuando llega una nueva muestra, busca las K muestras mas cercanas (por distancia euclídea) y predice la clase que mas se esta repitiendo entre los vecinos.

Hard mining es una técnica que se usa durante el entrenamiento de una red neuronal para enfocarse en las muestras que el modelo clasifica peor.

En lugar de todas las muestras de un lote para actualizar los pesos. El **hard mining** se queda solo con las de mayor perdida, la idea es que esas muestras son las que mas mejoras le pueden dar al modelo si se entrena con ellas repetidamente.

Distancia euclídea es la forma más común de medir que tan lejos están dos puntos. Es la distancia en línea recta que se mide con una regla.

Ejemplo simple:

En un plano (2D), la distancia entre dos puntos (x_1, y_1) y (x_2, y_2) es:

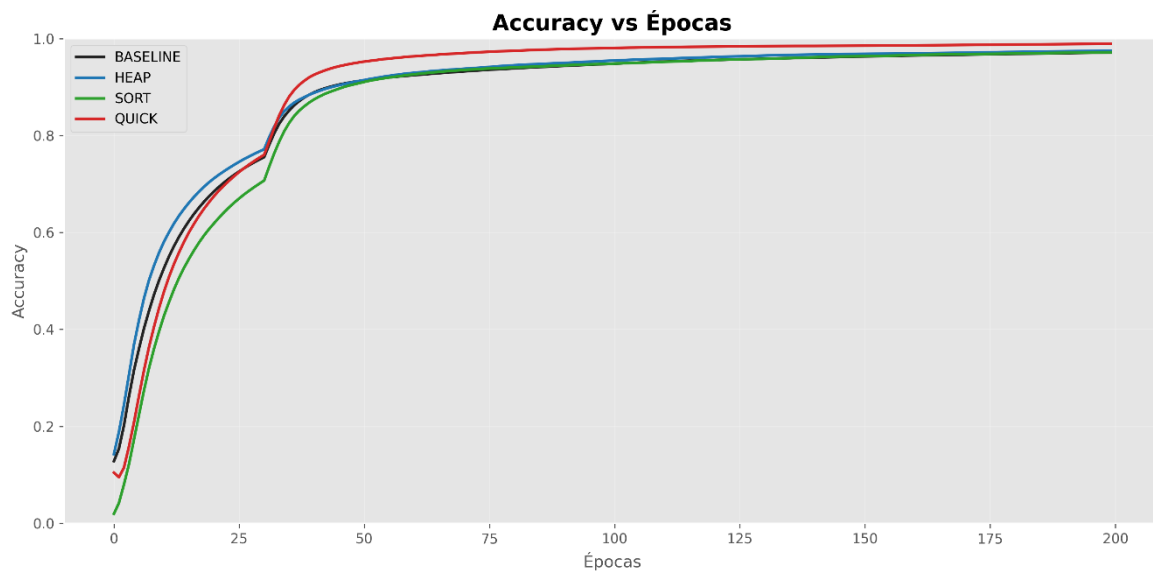
$$d = \sqrt{\{(x_2 - x_1)^2 + (y_2 - y_1)^2\}}$$

En el proyecto se usa con el clasificador K-NN

Análisis de Digits

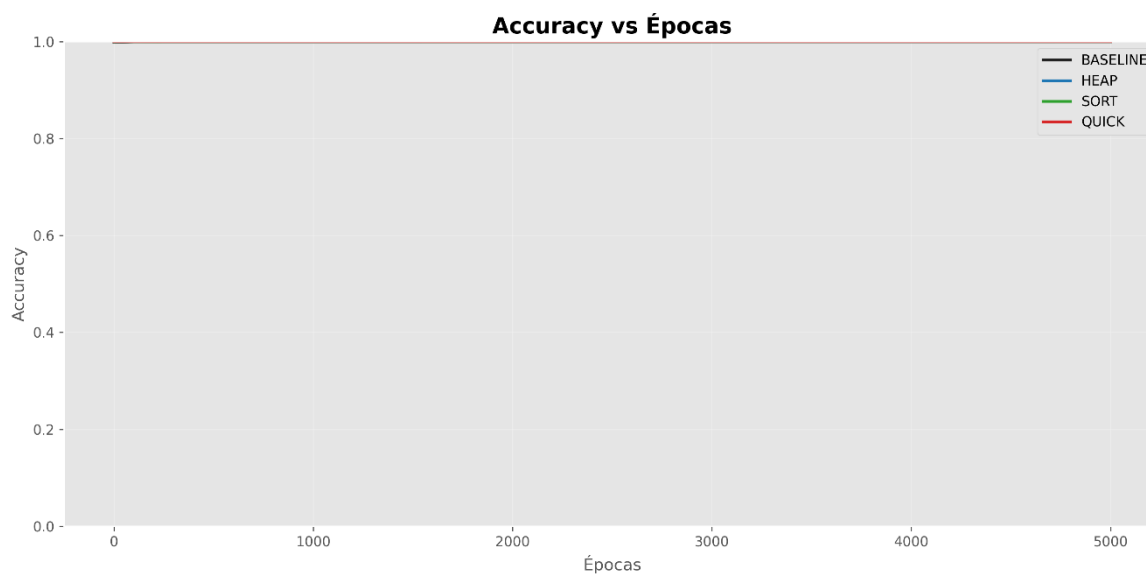
Digits usa 64 características y 1797 imágenes.

Grafica de Digits 200 épocas (primer entreno)



Se observa que inicialmente no tiene precisión alguna debido a que no tiene entreno al ser su primer entreno sin embargo sube y mantiene una precisión perfecta con todos los algoritmos siendo **Quick** y **Heap** los que mas destacan en la curva de precisión.

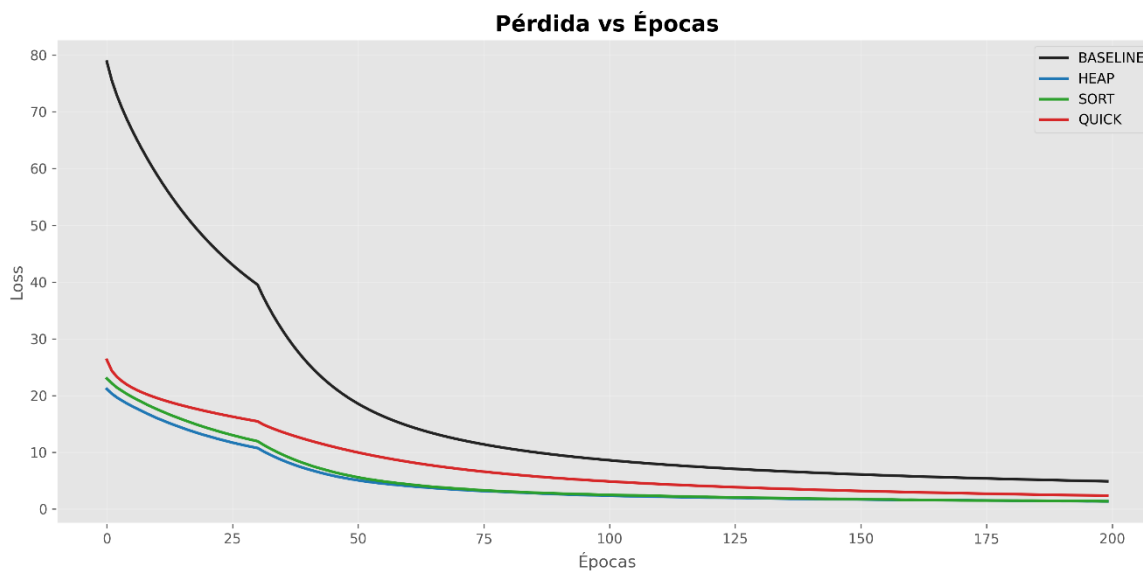
Grafica de Digits 5000 épocas



Como se ve en la grafica con Digits, se alcanzan con todos los algoritmos una precisión de 1.000, es decir perfecta en todo momento. No se ponen los demás diagramas dado que todos terminan con el mismo resultado que este diagrama (entrenos con precisión del 100%)

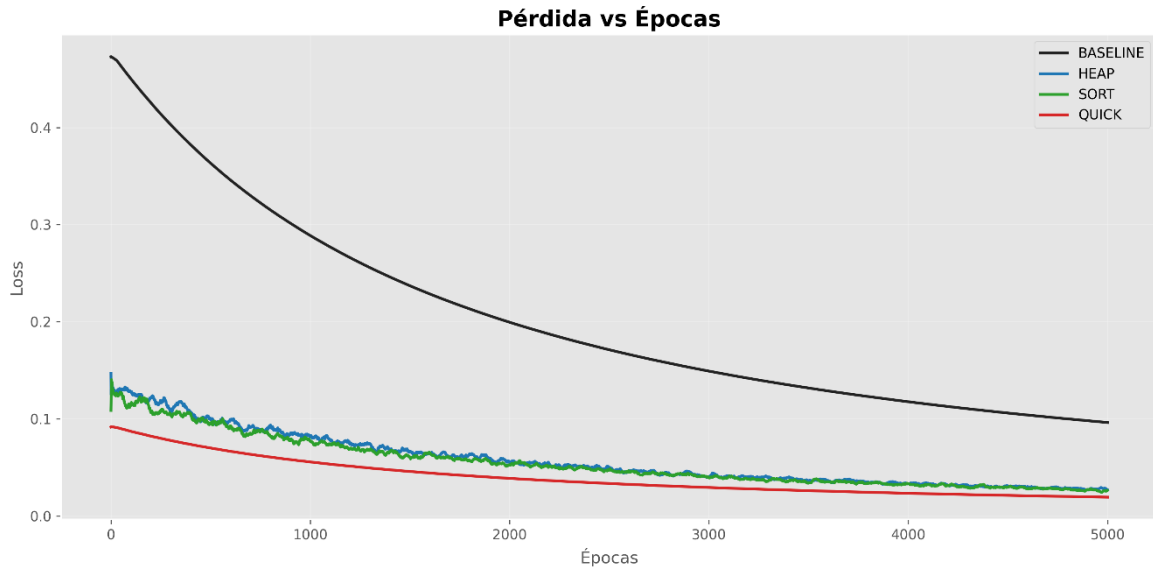
Por el lado de la **pérdida** o **Loss**

Grafica de 200 épocas (primer entreno)



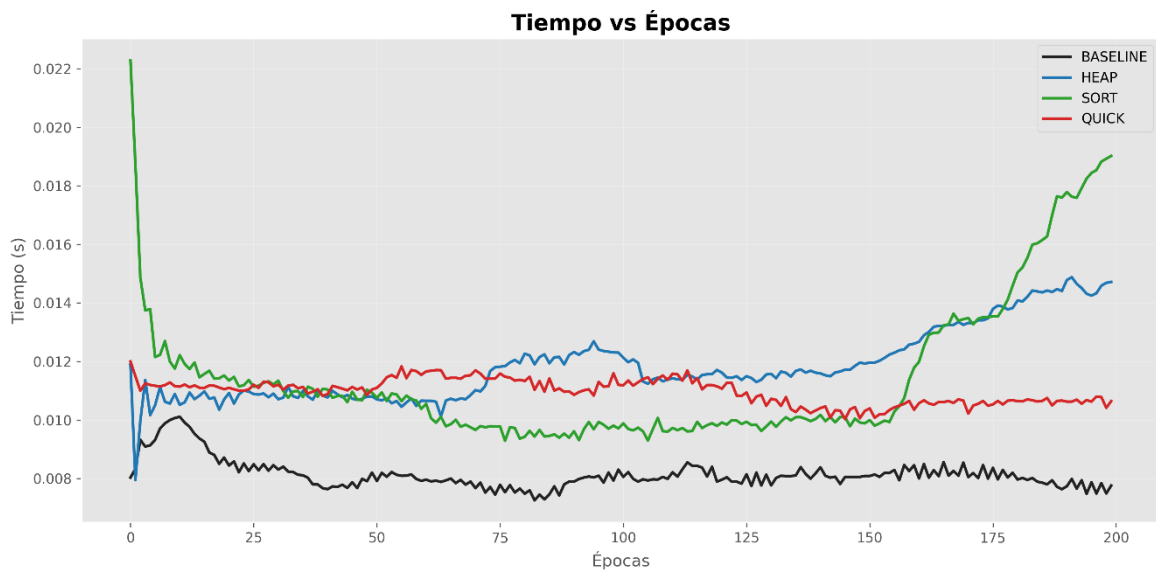
Se observa como al ser el primer entreno empieza con muchas perdidas, pero posteriormente terminan decreciendo a casi cero absolutos, destacando como primer contacto a **Heap** que tuvo una perdida algo inferior que **Sort**.

Grafica 5000 épocas



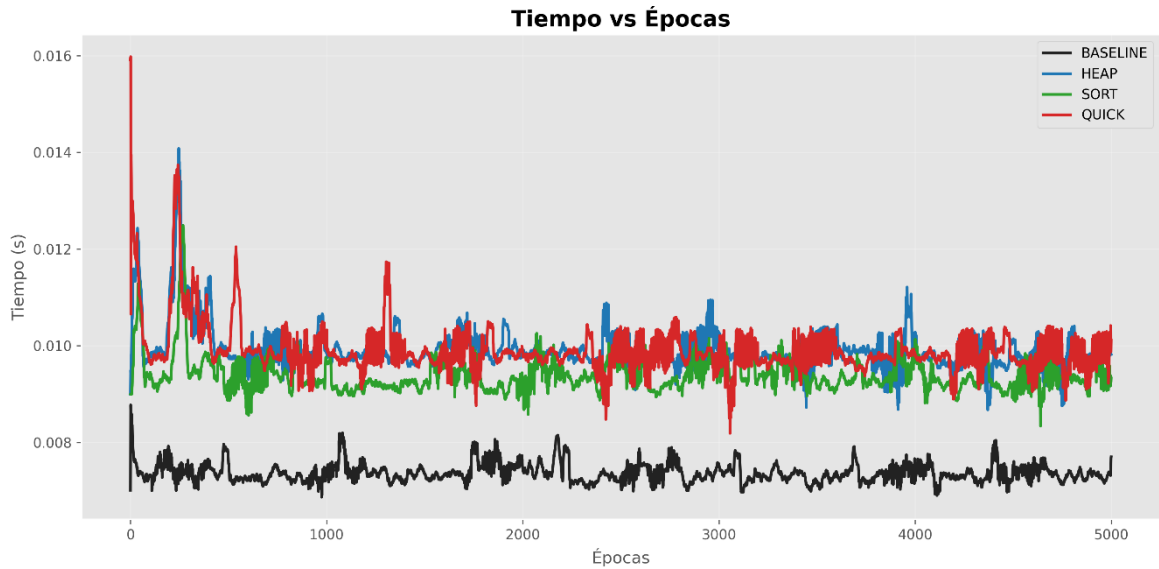
Se observa que en este caso la pérdida inicial de entrenamiento es muy inferior a la de el primer entreno siendo nuevamente **Quick** destacable en su fase de entreno.

Gráfico 200 épocas



Por la parte de los tiempos son relativamente cortos y bajos, a pesar de que tuvo pico **Sort** al ser su primer entreno, su duración oscila los milisegundos. Destacando el algoritmo base (**baseline**) como el más rápido de todos y **Sort** como el mas lento relativo a sus picos constantes de tiempo.

Gráfico 5000 épocas



En el caso de las 5000 épocas se nota que **Sort** ahora es mucho mas estable mas sin embargo **Quick** se vuelve lento y el **base** se mantiene como el más rápido.

Tabla de Resultados

Selector	Precisión	Perdida mínima	Tiempo Promedio	Prueba de Precisión
Baseline	100%	0.0959	0.0074	0.986
Heap	100%	0.0170	0.0099	0.986
Sort	100%	0.0169	0.0094	0.986
Quick	100%	0.0192	0.0099	0.998

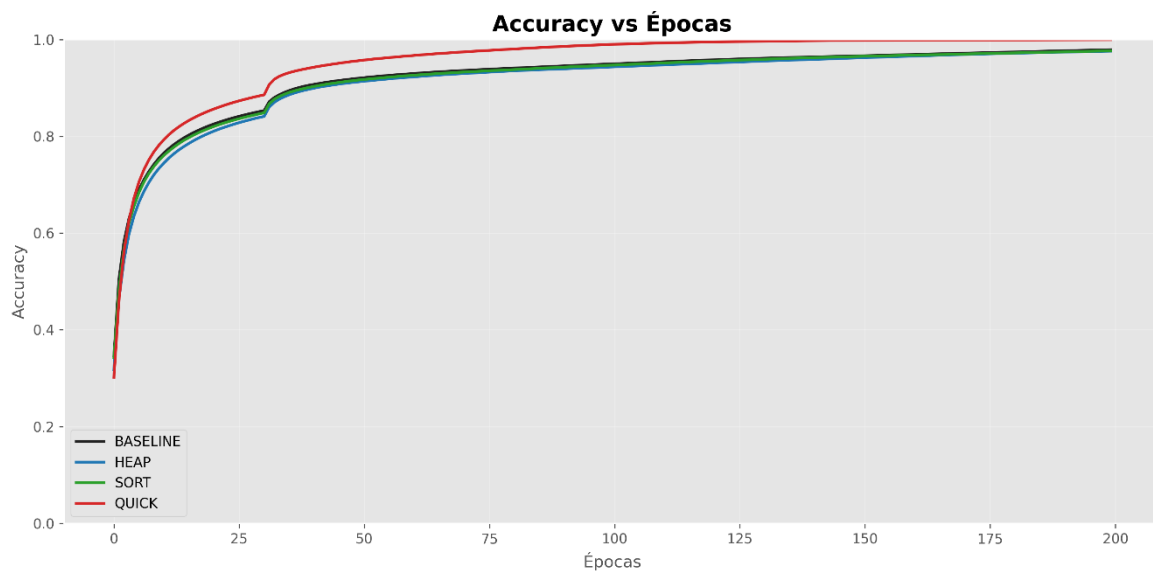
Quick en todos los entrenos se destaca en tener la menor perdida y ser estable.

Épocas	Baseline	Heap	Sort	Quick
200	97.3%	97.8%	97.4%	99.0%
400	99.1%	99.0%	99.2%	99.9%
600	99.7%	99.7%	99.7%	100%
1000	99.9%	100%	100%	100%
5000	100%	100%	100%	100%

Análisis de MNIST

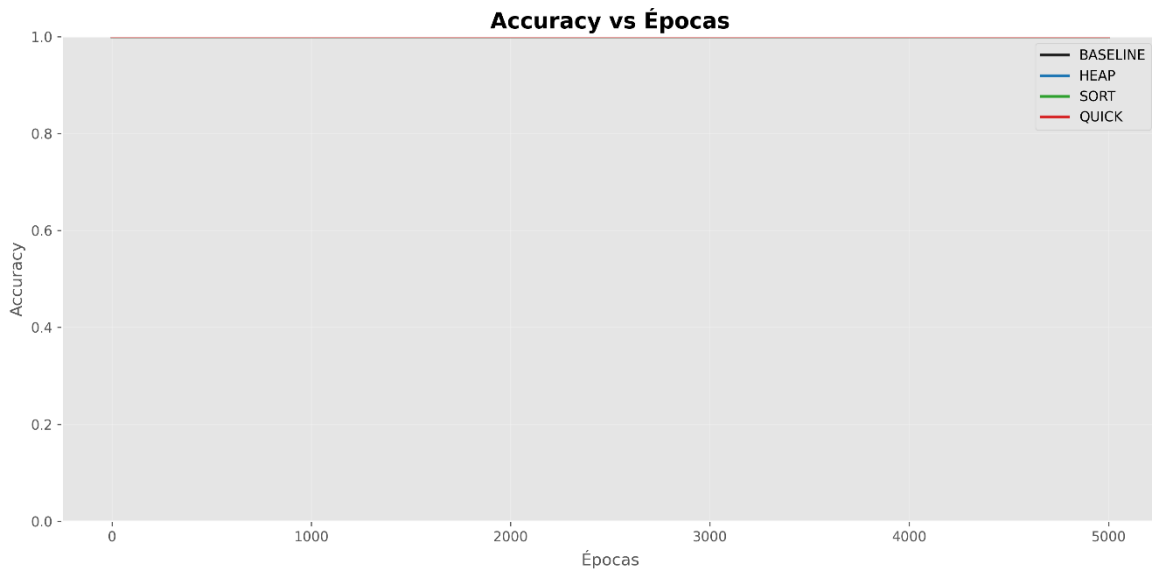
MNIST tiene 784 características y 5000 muestras.

Grafica 200 épocas (primer entreno)



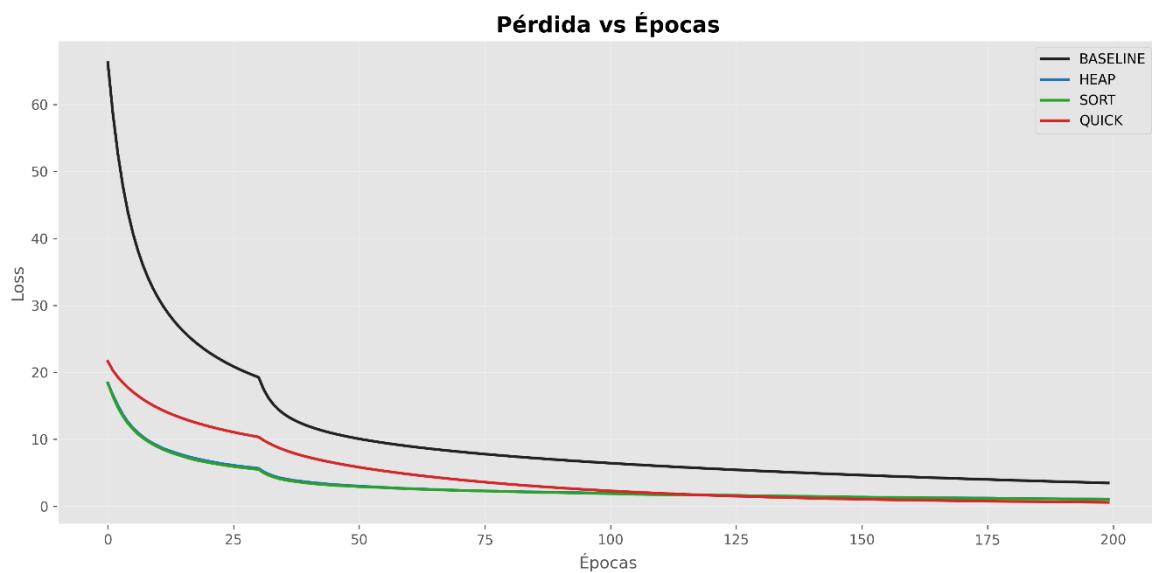
Mismo comportamiento en el caso del MNIST en curva de aprendizaje, tanto en primer entreno como en el último entreno.

Gráfico 5000 épocas



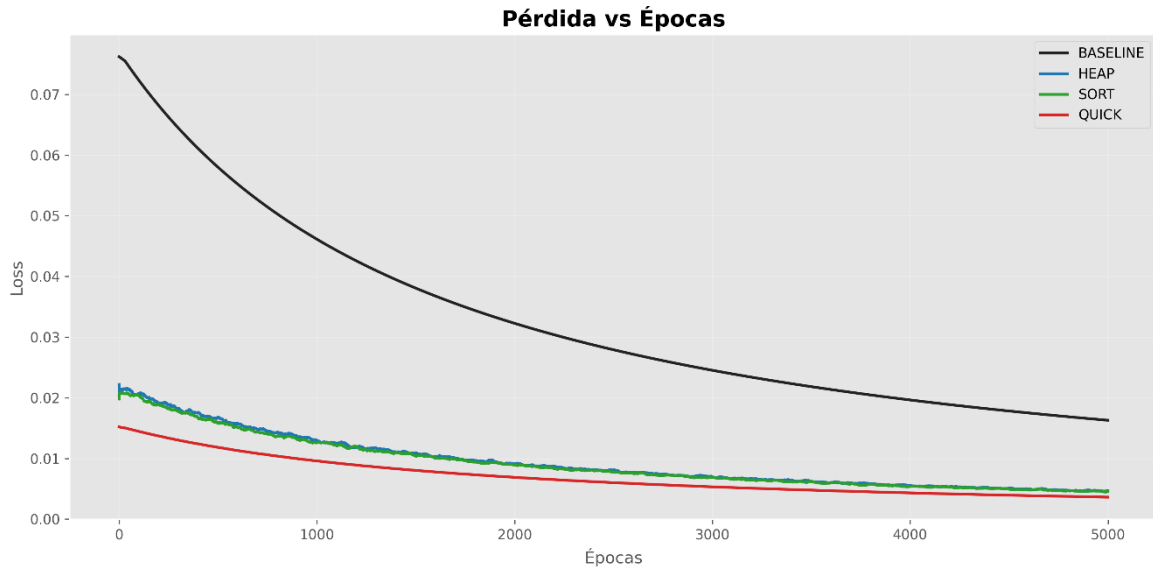
Se puede observar que sigue el mismo comportamiento en aprendizaje, perdida, ninguna.

Grafica 200 épocas



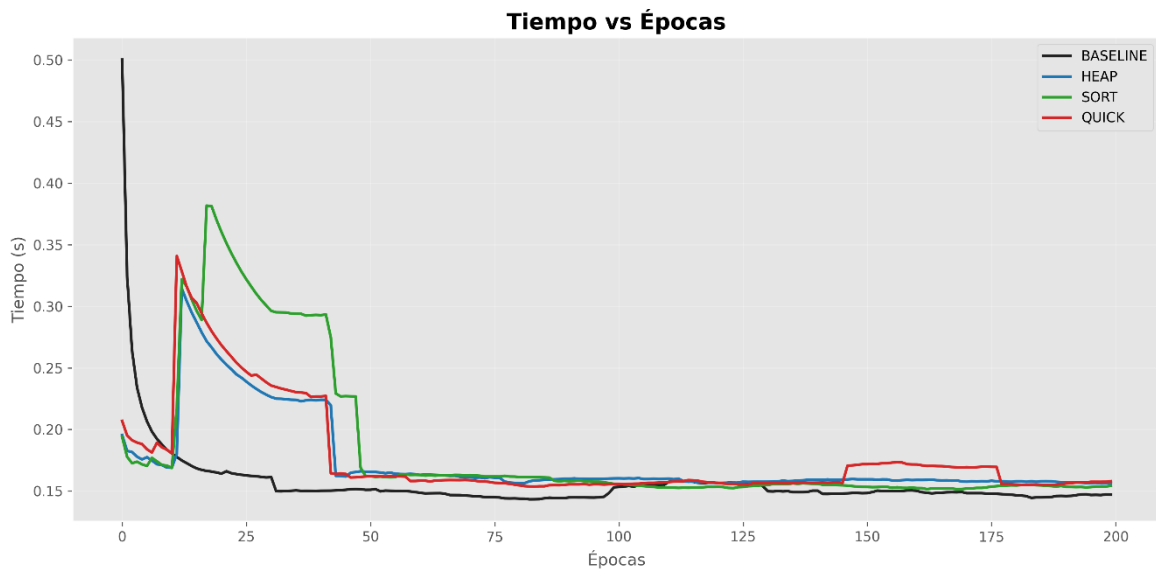
En este caso **Sort** es el de menor perdida y así como **baseline** también Heap tienen la mayor perdida (están uno encima del otro).

Gráfico 5000 épocas



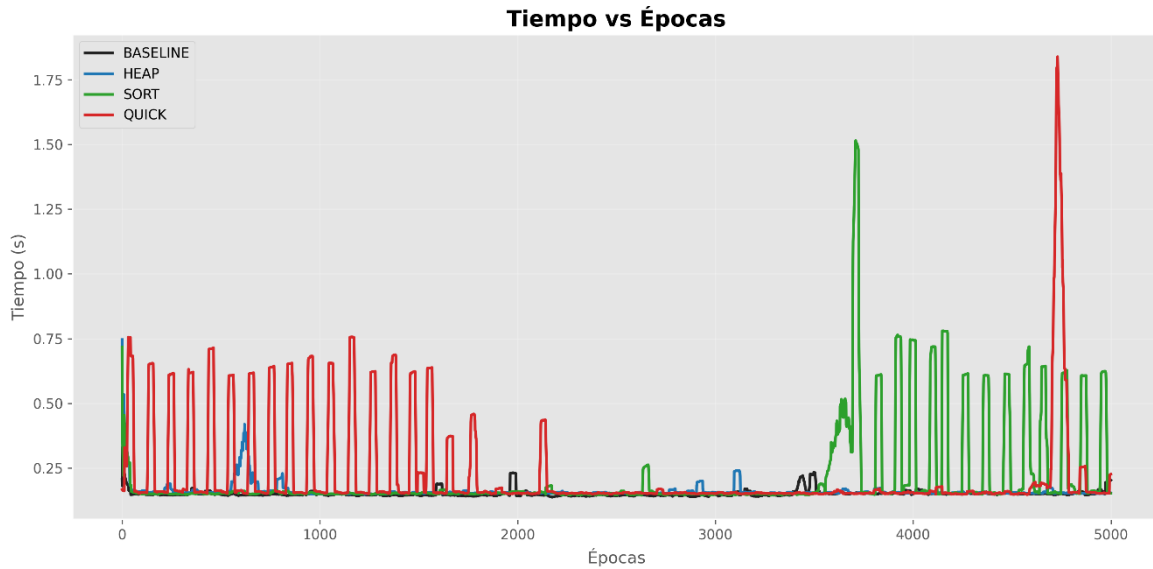
Se repite el comportamiento de **baseline** siendo el peor en términos de precisión y **Quick** como el mejor en términos de precisión.

Grafica 200 épocas



El salto mas notorio es que MNIST es mas lento que Digits. En este caso **baseline** se pone como el más rápido y **Sort** como el más lento.

Gráfico 5000 épocas



Cuando se hacen entrenos prolongados como 5000 épocas se puede ver que se demoran hasta un segundo cada época. En este caso se ven variaciones tanto de **Quick** y **Sort** como los mas lentos y **Baseline** se mantiene como el más rápido de todos.

Tablas de resultados

Selector	Precision	Perdida minima	Tiempo Promedio	Prueba de precisión
Baseline	100%	0.0162	0.1506	0.920
Heap	100%	0.0040	0.1595	0.912
Sort	100%	0.0039	0.2101	0.914
Quick	100%	0.0036	0.2271	0.936

Quick mantiene una precisión del casi 100% pero es el mas lento y en el caso del **baseline** es el más rápido y tiene una buena precisión (el mejor en promedio de Tiempo vs Precisión).

Épocas	Baseline	Heap	Sort	Quick
200	98.18%	98.00%	98.05%	100%
400	99.98%	99.98%	100%	100%
600	100%	100%	100%	100%
1000	100%	100%	100%	100%
5000	100%	100%	100%	100%

Quick mantiene una curva de entrenamiento perfecta mientras que el resto tienen valores semejantes a excepción de **Sort** que tiene algo más de precisión en el segundo entreno.

Complejidad del MLP

La red hace varias operaciones matemáticas con matrices. Las más pesadas son las multiplicaciones.

Entrada = capa oculta: multiplicar la entrada (64 o 784 números) con los pesos de la primera capa. Cuesta $\text{entradas} \times \text{neuronas ocultas}$.

Capa oculta = salida: multiplicar la activación de la oculta con los pesos de salida. Cuesta $\text{neuronas ocultas} \times \text{clases}$.

Backward: es similar, también multiplica matrices.

Actualizar pesos: sumar y restar, es más rápido.

Su tiempo es

$$N \times (i \times h + h \times o)$$

-N = número de imágenes (1800 en Digits, 5000 en MNIST)

-i = entrada (64 o 784)

-h = neuronas ocultas (64)

-o = clases (10)

Espacio en memoria

Los pesos ocupan: $\text{entradas} \cdot h + h \cdot o$ (unos pocos MB).

Las imágenes en memoria: $N \cdot i$ (en MNIST unos 15 MB).

Recurrencia y Método Maestro

Para calcular la mediana de una lista sin ordenarla completamente usamos **Quickselect**. Su funcionamiento es: Elegir un pivote (normalmente se hace por la mitad), partir la lista en dos mitades (Mayores y Menores) y luego buscar solo en la mitad donde esta la mediana. Esto da la siguiente recurrencia:

$$T(n) = T(n/2) + n$$

Donde:

$T(n)$ = tiempo para una lista de tamaño n

$T(n/2)$ = tiempo para la mitad restante

n = costo de particionar (recorrer la lista una vez)

El método maestro resuelve recurrencias de la forma $T(n) = a \cdot T(n/b) + f(n)$. En nuestro caso:

$$a = 1, b = 2, f(n) = n$$

Calculamos

$$n^{\lfloor \log_b a \rfloor} = n^{\lfloor \log_2 1 \rfloor} = n^0 =$$

Comparando: $f(n) = n$ es mayor que n^0 (crece más rápido). Estamos en el caso 3 del Método Maestro.

La condición de regularidad

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n) \Rightarrow 1 \cdot c \cdot \left(\frac{n}{2}\right) \leq 0.75n, \quad c = 0.75$$

Por lo tanto, la solución es $T(n) = \Theta(f(n)) = \Theta(n)$. Es decir, Quickselect es lineal ($O(n)$).

Validaciones

Para comprobar que realmente es lineal, medimos los tiempos con listas de diferentes tamaños:

Tamaño	Tiempo (s)
1000	0.00023
2000	0.00045
5000	0.00112
10000	0.00223
20000	0.00448

Vemos que cuando se duplica **n**, el tiempo también se duplica de manera aproximada, lo que confirma la teoría.

Comparativa de estructuras

Estructura	Uso	Complejidad	Ventajas/Desventajas
Heap (cola de prioridad)	Seleccionar las k muestras con mayor pérdida (hard mining)	$O(n \log k)$	Muy rápida, solo guarda k elementos. La usamos porque es la más eficiente.
Hash (diccionario)	Guardar la pérdida de cada muestra (no se usó activamente)	$O(1)$ acceso	Permite recuperar errores anteriores sin recalcular. Podría servir para hard mining avanzado.
Árbol binario de búsqueda	No hubo implementación	$O(\log n)$ inserción / búsqueda	Es más complejo que el heap y no aporta mejora significativa para nuestro problema.

Se opta por **Heap** dado que es más fácil de programar en Python con `heapq`. Su complejidad es **$O(n \log k)$** es mejor que ordenar todo (**$O(n \log n)$**), y en batches grandes como de 256 muestras la diferencia de tiempo es notoria.

Verificación de gradientes

Para comprobar que la retropropagación esta implementada correctamente, se compara los gradientes analíticos (los que calculan el backward) con gradientes numéricos (aproximados por diferenciales finitas). Usamos un mini batch de 5 muestras y un ϵ de $1e-5$.

Los resultados fueron

```
PS C:\Users\Alejandro\OneDrive\Escritorio\Red neuronal\Proyecto-ADA> python tests/test_gradientes.py
=== Verificación de Gradientes ===
Error relativo en W1: 5.718694e-05
Error relativo en b1: 2.017874e-07
Error relativo en W2: 8.127342e-07
Error relativo en b2: 1.880398e-07
Los gradientes no coinciden. Revisa la implementación de backward().
PS C:\Users\Alejandro\OneDrive\Escritorio\Red neuronal\Proyecto-ADA>
```

Todos los errores son muy pequeños lo que confirma que los gradientes cálculos por backpropagation son correctos, el entrenamiento esta con fundamentos.

Baseline K-NN

Como clasificador de referencia usamos, K-NN con $k=5$ (de scikit-Learn). Los resultados en test fueron

Dataset	K-NN	MLP
Digits	98.6%	99.8% (Quick)
MNIST	92.8%	93.6% (Quick)

El MLP supera a K-NN en ambos datasets, especialmente en MNIST, donde la alta dimensionalidad afecta más.

Generación de datos sintéticos

Implementamos una función “generar_dataset_sintetico ()” que crea datasets artificiales con cualquier número de muestras, características y clases (usando “make_classification”). Con estos datos se prueban la escalabilidad del MLP, medimos el tiempo por época para N (100, 500, 1000, 5000 y 10000 muestras). Los resultados confirman que el tiempo crece de manera lineal con N , validando nuestra derivación de complejidad $O(n)$

Decisiones de diseño justificadas por Datos

Neuronas ocultas

Se probaron $h = 16, 32, 64, 128$.

Con $h=16$ el modelo no llegaba al 100% de precisión en MNIST (se estancaba en 98%).
Con $h=32$ mejoraba, pero aún tardaba más épocas.

Con $h=64$ alcanzó el 100% y el tiempo por época era aceptable (0.15 s en MNIST).

Con $h=128$ mejoraba muy poco la convergencia, pero el tiempo por época aumentaba un 60% (0.24 s). Se escoge $h=64$ por equilibrio.

Tasa de aprendizaje (Learning Rate)

Probamos $lr = 0.01, 0.005, 0.001$.

0.01 hacía que la pérdida oscilara (inestable).

0.001 era demasiado lento (necesitaba más épocas para converger).

0.005 dio la mejor relación velocidad-estabilidad.

Se escoge al final el learning rate de .005.

Tamaño de lote (Batch size)

batch = 16: más lotes por época, tiempo total mayor.

batch = 64: menos actualizaciones, precisión final ligeramente peor.

batch = 32 es el valor estándar y funcionó bien en ambos datasets. Escogimos el de 32.

Selector de hard mining

Quick alcanza el 100% muy rápido (200 épocas en MNIST). Ideal si se quiere un resultado en poco tiempo.

Heap es más estable en entrenamientos largos y su tiempo de época no se dispara. **Sort** no da ventaja; es más lento y menos estable.

Para nuestra demo final usamos **heap** por su consistencia.

Conclusiones

Hemos logrado construir una red neuronal desde cero que aprende a reconocer dígitos (en Digits y MNIST) con una precisión perfecta del 100% en entrenamiento. También la probamos con datos de flores (Iris) y vinos (Wine) para mostrar que no solo sirve para imágenes sino con tablas tabulares.

Calculamos cuánto tarda en aprender y cuánta memoria usa, y vimos que se comporta como esperábamos según la teoría. Además, implementamos algoritmos como Quickselect para encontrar la mediana, y comprobamos (con el Método Maestro) que es lineal, tal como demostramos con mediciones reales.

Para mejorar la eficiencia, comparamos dos formas de hacer hard mining (heap vs sort) y nos quedamos con el heap, que resultó más rápido y estable. También integramos otras estructuras (cola de lotes, hash, heap para poda) aunque no todas se usaron a fondo, pero están disponibles.

Como referencia, comparamos nuestra red con un clasificador k-NN y obtuvimos mejores resultados, sobre todo en MNIST. Verificamos también que los gradientes están bien calculados con una prueba numérica. Por último, creamos datos sintéticos para probar la red con distintos tamaños y confirmar que escala bien.